

Start from what people search for.

You type a few keywords. Seven small steps later you know which video topics are rising on Google and already working on YouTube.
This guide walks through every step with a real example.

You have a hunch. Check it before you film.

A video costs you days. This pipeline takes the words you would use to describe your niche and tests them against two kinds of public data, so you film the topic with the best odds.

[Google](#)

Test 1. Is it rising?

Are more people searching Google for this than a year ago? Or has the crowd already moved on?

[YouTube](#)

Test 2. Is it repeatable?

Has this topic given **many different channels** a hit lately? Or did it work once, for one creator?

A topic that passes both gets the verdict [make it](#). This is a real result, from the seed keywords "opencode", "codex" and "claude code":

TOPIC	REPEATABILITY	CHANNELS	SEARCHES / MONTH	VERDICT
OpenCode	4.36	15	110,000	make it
Codex app on Windows and Mac	3.68	10	1,900	make it
Claude Code	3.44	13	550,000	make it
Codex AI coding tool	2.22	8	368,000	make it
Codex Python	1.94	13	210	rising, but not repeatable
Codex Gigas, and 17 more	-	-	6,600	skipped: unrelated

THE PRINCIPLE BEHIND IT

Two independent kinds of evidence that agree beat any amount of one kind.

Google search and YouTube views are measured by different systems, from different behaviour. A topic can fool one of them. It rarely fools both.

Seven steps. Each one asks one question.

Every step takes the previous step's answer, does one small thing, and saves what went in and what came out. The colour tells you where the answer comes from.

1	Google	Keyword ideas	What do people search for around my seed keywords?
2	Rule	Search trends	Is each keyword rising, flat or shrinking?
3	Rule	Rising keywords	Which ones are rising, with enough searches to trust?
4	LLM	Group into topics	Which video topics are those keywords about?
5	Rule	Choose topics	Which topics are worth a YouTube search?
6	YouTube	Repeatability	Has each topic worked for many channels?
7	Rule	Scoreboard	Which topics are repeatable and rising?

The same run, as a funnel



Real numbers from the run on the previous page. Most of the work is throwing things away, cheaply, before the one step that costs YouTube quota.

FOUR KINDS OF STEP

Google and **YouTube** steps fetch facts. **Rule** steps are plain arithmetic you can check by hand. **LLM** steps use an AI model only for language judgement, such as "are these two phrases the same topic?", and never for numbers.

Pick seed keywords like you'd aim a flashlight.

A seed keyword only tells Google where to look. It does not have to be the answer. The later steps throw away what doesn't fit.

RULE OF THUMB

WHAT IT MEANS

Short and noun-like

One to three words that name a thing: **claude code** , **ai agents** . Not a whole question. Google does the expanding for you.

One theme per run

Two or three seeds the same viewer would care about. Start a new run for the next theme.

Avoid words with two lives

See the example below.

Where to find them

The search terms in your YouTube Studio analytics, the tools and problems of your niche, and the keywords a previous run turned up.

A real example of a seed gone wrong

The run in this guide used the seed **codex** , meaning the AI coding tool. But a codex is also a medieval manuscript and a Warhammer rule book. Google blends all seeds into one answer, and this is who got the space:



From the file 01_a_seeds_to_keywords.json of the run. One ambiguous seed crowded out the other two.

WHAT THIS TEACHES

A bad seed rarely gives a wrong verdict. It gives a blind spot.

The manuscripts were caught and skipped in step 4, so the scoreboard is still right. The cost is what you never saw: almost no keywords about Claude Code. The fix is a sharper seed, such as **openai codex** , or a run for each tool.

What do people search for on Google?

Your seed keywords go to Google's Keyword Planner, the tool advertisers use. It answers with the related things people type into Google, and how many times a month, for each of the last 48 months.

GOES IN

opencode
codex
claude code

Three seed keywords, typed by you.



COMES OUT

claude code	550,000 a month
codex	368,000 a month
opencode	110,000 a month
codex gigas	18,100 a month
open code download	880 a month

and 1,058 more, each with 48 months of history.

WHY IT WORKS

A search is an honest signal.

Nobody searches to look smart. People type what they actually want, when nobody is watching. And advertisers spend real money based on these numbers, so Google has a reason to keep them right.

WHERE IT FALLS SHORT

- The numbers are rounded into steps (880, 1,000, 1,300 ...). Read them as sizes, not exact counts.
- It is Google search, not YouTube search. It shows that people want an answer, not yet that they want a video.
- Your seeds go to Google together and come back as one mixed list. Google does not give each seed a fair share. In this run 825 of the 1,063 keywords had "codex" in them and only 1 had "claude code", so the run learned almost nothing about Claude Code. Page 4 shows how to avoid that.

TRY IT `uv run python -m steps.keyword_ideas "claude code"`

SAVED AS 01_a_... and 01_b_keyword_ideas.json

Is each keyword rising, flat or shrinking?

For every keyword, compare the last 12 months with the 12 months before. But don't compare the averages. Compare the **floor**: the average of the three quietest months of each year.

GOES IN

codex	48 months
codex gigas	48 months
codex sinaiticus	48 months

Each keyword with its monthly searches.

COMES OUT

codex	floor 38,033 → 103,500 rising
codex gigas	floor 13,900 → 13,900 flat
codex sinaiticus	floor 12,100 → 9,300 shrinking

Up 20% or more is rising. Down 20% or more is shrinking. Nothing a year ago is new.



Monthly Google searches for "codex". The blue dots are the three quietest months of each year, the blue line is their average: the floor.

WHY IT WORKS

Measure the quiet months, not the loud ones.

One news week can double a year's average. It cannot lift the three quietest months. The floor is the demand that is there on a boring day: the audience still around when your video comes out.

WHERE IT FALLS SHORT

- It needs two years of history. Brand-new topics are labelled "new" instead of measured.
- The floor reacts slowly. A topic that took off two months ago can still read as flat.
- The 20% line is a judgement call. Google's rounding makes smaller changes unreadable anyway.

TRY IT `uv run python -m steps.search_trends` (needs no keys)

SAVED AS `02_search_trends.json`

Which keywords are rising, with enough searches to trust?

A simple filter with three tests. A keyword goes on when it gets at least 200 searches a month, its trend is rising or new, and its whole year did not fall by 20% or more.

GOES IN

1,063 keywords, each with the trend from step 2.



COMES OUT

58 keywords go on to step 4.

1,005 are dropped. Each one keeps the reason it was dropped, so you can check any keyword.

THE ARITHMETIC. A KEYWORD IS DROPPED AT THE FIRST TEST IT FAILS, IN THIS ORDER.

1,063 keywords came from Google in step 1

-	891	Too small. Under 200 searches a month.	"open code software": 20 a month	172 left
-	95	Big enough, but flat. Floor moved under 20%.	"codex gigas": floor unchanged	77 left
-	19	Big enough, but shrinking. Floor fell 20%+.	"codex sinaiticus": floor down 23%	58 left
-	0	Rising, but the whole year fell 20% or more. None in this run.		58 left
=	58	Go on. Rising or new, with 200 or more searches a month.	"claude code", "opencode"	

WHY IT WORKS

Small numbers lie.

A keyword that goes from 10 to 30 searches is "up 200%" and means nothing. Demanding a minimum size removes the noise. And filtering here is free, while every topic that reaches step 6 costs YouTube quota.

WHERE IT FALLS SHORT

- 200 searches is a judgement call. A tiny niche may need a lower bar, and you can change it in config.py.
- It does not know what your channel is about. Rising keywords about medieval manuscripts pass too. Removing them is the next step's job.

TRY IT `uv run python -m steps.rising_keywords` (needs no keys)

SAVED AS `03_rising_keywords.json`

Which video topics are those keywords about?

An AI model reads the rising keywords and groups the ones that are really the same video. It also flags the groups that are not worth a YouTube search: unrelated to your seeds, too broad, or just a brand name.

GOES IN

opencode
open code
open code download
codex gigas devil
warhammer 40k codex
and **53** more rising keywords.



COMES OUT

OpenCode · 4	search YouTube for keywords	"opencode"
Codex AI coding tool · 4 keywords		"codex ai"
Codex Gigas · 3 keywords		skipped: unrelated
Warhammer 40k codex books · 15		skipped: unrelated
The Codex · 1 keyword		skipped: too broad

23 topics, of which **18** are skipped.

WHY IT WORKS

A video answers a need, not a phrase.

People word one need in many ways. Grouping them means one YouTube search instead of nine. Telling the AI tool "Codex" from the manuscript "Codex Gigas" takes language judgement, which no simple rule can do. The guardrail: the model may only use keywords from the list. Anything it invents is thrown away.

WHERE IT FALLS SHORT

- It is a judgement, so it can be wrong, and a different model may group differently. The reasons are saved, so check them.
- It is the one step in this pipeline you cannot verify with a calculator.

TRY IT `uv run python -m steps.group_into_topics` (one small LLM call)

SAVED AS `04_group_into_topics.json`

Which topics are worth a YouTube search?

The topics that were not skipped are ranked by how many searches a month their biggest keyword gained in a year. The top 10 go on.

GOES IN

23 topics, 18 of them already skipped.



COMES OUT

Claude Code	+405,114 searches a month
Codex AI coding tool	+347,241
OpenCode	+105,511
Codex app on Windows and Mac	+2,074
Codex Python	+41

All 5 go on to YouTube.

WHY IT WORKS

Spend the scarce thing where the demand grew most.

YouTube gives you 10,000 quota units a day and one search costs 100. So the pipeline ranks by searches **gained**, in absolute numbers. Growth of 2,000% from almost nothing is still almost nothing.

WHERE IT FALLS SHORT

- It favours big topics. In a broad run, a small niche topic may never make the top 10. Run narrower seeds to see it.
- A topic is judged by its single biggest keyword.

TRY IT `uv run python -m steps.choose_topics` (needs no keys)

SAVED AS `05_choose_topics.json`

Has each topic worked for many channels?

One YouTube search per topic. Every result is compared with what **its own channel** normally gets. A video at 4 times its channel's normal is a hit. Every hit adds to the topic's score, and newer hits, and hits from channels near your size, add more.

GOES IN

opcode
codex windows mac
claude code
codex ai
codex python

Five YouTube searches, one per topic.

COMES OUT

opcode	19 hits on 15 channels · 4.36
codex windows mac	14 hits on 10 channels · 3.68
claude code	22 hits on 13 channels · 3.44
codex ai	12 hits on 8 channels · 2.22
codex python	14 hits on 13 channels · 1.94

The last number on each line is the topic's score.

WHAT "REPEATABLE" MEANS

A topic is repeatable when it passes two tests at once: its hits come from **6 or more different channels**, and its **score is 2.0 or more**. A score of 1.0 is worth one brand-new hit from a channel exactly your size. Four topics pass. "codex python" has enough channels, but its score of **1.94** is just under the bar. The next page works through one score by hand.

WHY IT WORKS

Divide by the channel, then demand many channels.

A video's views come from the channel, the topic, the packaging and luck. Dividing by what the channel normally gets cancels the channel. Asking for hits on many unrelated channels cancels packaging and luck, because those differ from channel to channel. What is left is the topic.

WHERE IT FALLS SHORT

- It only sees the videos YouTube's search returns. Most flops on the topic stay invisible.
- The bars (4 times normal, 6 channels, the score) are judgement calls. You can change them.
- It measures the topic, not you. A repeatable topic still needs a good title, thumbnail and video.

TRY IT `uv run python -m steps.score_repeatability "claude code"`

SAVED AS `06_score_repeatability.json`

How "opencode" got its score of 4.36.

1 Search, and set aside what can't be compared

YouTube returns up to 50 videos for "opencode". Videos older than 2 years, or with under 1,000 views, are set aside.

2 Is each video a hit? $\text{views} \div \text{its channel's normal} \geq 4$

VIDEO	VIEWS	CHANNEL'S NORMAL	RESULT
"The Complete OpenCode Course" · Superposition	1,097	225	4.9x hit
"OpenCode Full Tutorial" · Eric Tech	131,849	6,882	19.2x hit

A video at 2 times its channel's normal is not a hit and adds nothing. This search found 19 hits, on 15 different channels.

3 How much does each hit count? $\text{freshness} \times \text{closeness}$

Freshness halves every 180 days: today 1.0, six months 0.5, a year 0.25. **Closeness** is 1.0 for a channel your size, 0.5 for one 10 times bigger or smaller, 0.33 for 100 times. In this run your channel's normal is about 140 views.

HIT	AGE → FRESHNESS	CHANNEL SIZE → CLOSNESS	COUNTS FOR
OpenCode Course	31 days → 0.89	1.6 times yours → 0.83	$0.89 \times 0.83 = 0.74$
OpenCode Full Tutorial	76 days → 0.75	48 times yours → 0.37	$0.75 \times 0.37 = 0.28$
The oldest of the 19 hits	339 days → 0.27	307 times yours → 0.29	$0.27 \times 0.29 = 0.08$

The 19.2x hit counts for less than the 4.9x hit. How big a hit was does not matter. How recent it is, and how much the channel is like yours, does.

4 Add up all 19 hits

$$0.74 + 0.35 + 0.32 + 0.28 + 15 \text{ smaller ones} = 4.36$$

5 The verdict: two tests, both must pass

TOPIC	CHANNELS WITH A HIT (6 OR MORE)	SCORE (2.0 OR MORE)	VERDICT
opencode	15 pass	4.36 pass	repeatable
codex python	13 pass	1.94 fail	not repeatable

The channel test stops one channel with many hits from carrying a topic. The score test stops old hits from giant channels from carrying it.

Which topics are repeatable and rising?

The last step puts the two tests side by side, one row per topic, and writes the verdict. It also saves the result as a spreadsheet.

GOES IN

The topics, their Google keywords and their YouTube scores.



COMES OUT

OpenCode

make it

Claude Code

make it

Codex Python rising, but not repeatable

Codex Gigas skipped: unrelated

4 to make, 1 rising but not repeatable, 18 skipped.

WHY IT WORKS

Every verdict carries its reason.

A topic is never just rejected. It is "rising, but not repeatable" or "skipped: unrelated". That tells you what to do next: wait, rephrase, or move on. And because every step saved its work, you can walk back to the exact number behind any verdict.

WHERE IT FALLS SHORT

- "Make it" means the odds are good. It is not a promise. Your title, thumbnail and video still decide the result.
- The data moves. A verdict is a snapshot of today, so run it again before you commit weeks to a topic.

TRY IT `uv run python -m steps.scoreboard` (needs no keys)

SAVED AS 07_scoreboard.json and scoreboard.csv

What this pipeline can't tell you.

It is a way to check your hunches with data. It is not an oracle. Knowing where it is weak is what lets you trust it where it is strong.

LIMIT

WHAT TO DO ABOUT IT

It only looks where you point it

It finds topics near your seeds. It cannot find a topic you had no words for. Pipeline 2 covers that: it starts from videos instead of words.

Google search is not YouTube search

People google some things they would never watch, and watch things they would never google. That is why step 6 checks YouTube separately.

It runs late on brand-new topics

Keyword data arrives monthly and the floor needs quiet months to rise. For something that launched last week, pipeline 2 is faster.

The thresholds are opinions

200 searches, 20%, 4 times normal, 6 channels. They are all named settings in config.py. Change one, run again, and see which verdicts move.

One country, one language

The default is the United States and English. Both are settings.

FIRST PRINCIPLE**A model you can't inspect, you can't improve.**

Every step saves a file with a summary, what went in and what came out. When a verdict surprises you, open the run folder and walk back, file by file, to the step that caused it. Then decide whether the pipeline is wrong or your hunch was.

When to use pipeline 2 instead

Use this pipeline when you already have words for what you want to cover. Use pipeline 2, from YouTube, when you want to know what is working in your niche right now, including topics you have not thought of.

RUN IT YOURSELF

One command.

Set it up once, then give it your seed keywords. A page opens in your browser that shows every step of the run.

```
cp .env.example .env
```

your YouTube and LLM keys

```
cp config.py.example config.py
```

your channel and your niche

```
uv run pipeline1_from_keywords.py "claude code" run it
```

Google Ads access takes a short application. The README in the repo walks through it.